

SyncSpace — Progress Tracker

Axlero Solutions · Project 1 · Advanced MERN Stack Engineering

Serah Joyce Rasquinha · audited against Milestone 5 (Accounts, Remote Execution, AI Assistant, Team Chat) and the replay coalescing fix · updated 14 August

How to read this

Every task below comes from the Axlero week-wise plan (and, where the blueprint is more specific, from the SyncSpace blueprint). The status colour tells you where you stand; the file column tells you exactly where to look — or where to start writing.

DONE	Built and working. Evidence exists in the codebase.
BONUS	Built, and it goes BEYOND what Axlero asked for this week. Mention these in reviews.
VERIFY	Built, but unproven or weak. You must test it, record it, or tighten it.
OPTIONAL	Not built. Optional / bonus / deployment. Will not fail a review.
NOT STARTED	Not built, and it IS required by the Axlero plan. This is your real backlog.

Where you actually stand after Milestone 5: Weeks 1, 2, 3 and 4 remain COMPLETE, and the whole board has grown two layers beyond the original Axlero plan since the last audit. Code execution moved off the local machine entirely — the sandbox that used to spawn gcc/javac/python3 as child processes is now a fallback of last resort, and every run by default goes to a remote provider chain (Judge0 CE first, Piston and paiza.io behind it), orchestrated with queueing, probing and one canonical result shape so the IDE panel never knows which service actually ran the code. An optional account layer landed on top of the anonymous workspace model: sign up, log in, and a dashboard lists every workspace you belong to for one-click re-entry — built as a parallel identity system that a workspace never has to know about, so the anonymous flow this project started with is completely untouched. The workspace itself gained a real-time team chat panel riding the existing Yjs pipeline, and the code pane gained an OpenAI-backed AI assistant (chat, explain, generate, debug, tests, optimize, convert, document) with a safe mock fallback when no API key is configured. A structural React error boundary now prevents any single bad render from blanking the whole app. And the one open risk flagged in the last audit — a runaway drag flooding the replay log — is now closed: same-user updates inside a 400ms window coalesce into one entry, and the per-room ceiling was raised 10x (5,000 → 50,000 updates) now that the transport genuinely does not care about session length. The Axlero week plan remains complete end to end, and every load-bearing addition since sits on top of it without touching the sync protocol.

Week 1 — Socket Infrastructure & UI Scaffolding

Goal: a client can connect, join a named room, and every other client in THAT room (and no other) receives what it sends. Plus a two-pane shell to hang it on.

Task	Status	File / Evidence	What is left / Notes
Set up Express HTTP server	DONE	backend/server.js	Running on :5000. Mounts the workspace REST API, the account/auth API, the execution API, and a central error handler.
Attach Socket.io to the HTTP server (with CORS for the Vite origin)	DONE	backend/server.js	CORS locked to CLIENT_ORIGIN from .env.
Room join mechanism (socket.join)	DONE	backend/services/socketService.js	Rooms are now WORKSPACES. The room a socket joins is

Task	Status	File / Evidence	What is left / Notes
			read from its signed token, not from anything the client sends.
Room ISOLATION - messages reach only that room	DONE	backend/services/socketService.js backend/services/realtime.js	Hardened. A socket can only reach ws:<id> where <id> came from its own token, so it CANNOT ask to join another workspace. Verified by test-workspace.mjs.
Track who is connected to each room	DONE	backend/services/realtime.js	room-info carries the full connected user list (username + role), not just a count.
Environment config + dev script	DONE	backend/.env backend/package.json	Adds JWT_SECRET and OPENAI_API_KEY (optional — the AI assistant degrades to a clear mock reply without it). MONGO_URI may be left blank.
React split-screen layout (whiteboard left, editor right)	DONE	frontend/src/pages/Workspace.jsx frontend/src/assets/index.css	Grid now flexes for a third column (admin panel) and a slide-out chat sidebar without disturbing the canvas/editor split.
Socket.io client connection from React	DONE	frontend/src/utils/socket.js	Sends a token in the handshake. SERVER_URL reads VITE_SERVER_URL (deploy-ready).
Room entry / routing into a session	DONE	frontend/src/pages/Landing.jsx frontend/src/pages/JoinWorkspace.jsx	Typing a room ID to get in was removed at Milestone 1. Signed-in users now have a second, faster door in via the Dashboard — see Milestone 5.
Live connection status indicator	DONE	frontend/src/pages/Workspace.jsx	Dot + peer chips, plus an admin crown and a policy chip.

Verdict: PASSED, and now hardened. Isolation is no longer a matter of trust: a socket is placed in ws:<id> using the ID inside its own signed token, so it cannot ask for another workspace. Dashboard.jsx (type-an-ID-to-enter) has been removed outright — the name was later reused for the signed-in account dashboard, an unrelated, safer feature (Milestone 5).

Week 2 — CRDT Integration & Canvas Engineering

Goal: the backend relays Yjs awareness (cursors, presence); the frontend draws on a Konva canvas and syncs the coordinates.

Task	Status	File / Evidence	What is left / Notes
Backend acts as a Yjs AWARENESS server	DONE	backend/services/socketService.js	awareness-update relayed to room peers - only for authenticated member sockets.

Task	Status	File / Evidence	What is left / Notes
Broadcast cursor positions between clients	DONE	frontend/src/hooks/useCollaboration.js frontend/src/components/Canvas.jsx	Cursor written to awareness on mouse move. Cursor coordinates are WORLD coordinates, so they land correctly at any zoom level on any screen.
Track connected users (presence list)	DONE	frontend/src/hooks/useCollaboration.js	peers[] from awareness, cross-checked against the DB member list.
Server keeps an authoritative Y.Doc per room	BONUS	backend/services/socketService.js	Unchanged. Still what lets a LATE joiner receive the full canvas state.
Integrate Konva.js into React	DONE	frontend/src/components/Canvas.jsx	The full drawing surface: tools, selection, transform, text, connectors, camera.
Drawing tool: FREEHAND LINES	DONE	frontend/src/components/Canvas.jsx	Pen tool. Points in a nested Y.Array so concurrent strokes append, not overwrite. Milestone-0 strokes still render.
Drawing tool: RECTANGLES	DONE	frontend/src/components/Canvas.jsx frontend/src/canvas/shapeDoc.js	DONE. Drag-to-create, live preview, negative-drag, min-size, full property schema.
Drawing tool: TEXT	DONE	frontend/src/components/Canvas.jsx	Drag-to-define a region, wrapping, auto-grow, double-click re-edit, empty-cancel. The overlay also tracks the zoom/pan camera.
Shapes toolbar: 5-10+ flowchart + geometric shapes	BONUS	frontend/src/components/Toolbar.jsx frontend/src/canvas/shapes.jsx	16 shapes + pen + a "Connectors & Arrows" group with 12 presets.
Generic shape abstraction (one create + render pipeline)	BONUS	frontend/src/canvas/ShapeNode.jsx frontend/src/canvas/shapeDoc.js	One record schema, one renderer switch, one write path. Connectors slot into the SAME array and inherit sync/undo/persistence for free.
Selection: bounding box, handles, multi-select	DONE	frontend/src/components/Canvas.jsx	Konva Transformer. Shift/Ctrl for multi-select. Connectors get their own endpoint/bend handles instead of a box.
Resize (corner/edge) + rotate, synced	DONE	frontend/src/components/Canvas.jsx	Scale baked into width/height on transform end. Circle/ellipse/star

Task	Status	File / Evidence	What is left / Notes
			centre-offset bug fixed via <code>isCentered()</code> .
Move / drag shapes, synced	BONUS	frontend/src/components/Canvas.jsx frontend/src/canvas/shapeDoc.js	Drags STREAM throttled (45ms) live commits under an undo-invisible origin - remote users watch the shape glide, yet the whole drag undoes as ONE step.
Delete (key / Backspace / toolbar / panel)	DONE	frontend/src/components/Canvas.jsx	All four paths remove from Yjs and clear selection. Deleting a shape never breaks connectors attached to it (cached fallback).
Fill / stroke / stroke-width / opacity / border style	DONE	frontend/src/components/PropertyPanel.jsx	Contextual panel; every change writes straight to Yjs so it is live. Also: lock, duplicate, bring forward / send backward.
Text formatting (fonts, size, B/I/U, align, line-height)	DONE	frontend/src/components/PropertyPanel.jsx	9 font families, colour picker, opacity, rotation.
Awareness: remote selection box + name	DONE	frontend/src/components/Canvas.jsx	Each peer's selection drawn in their own colour with their name. Connector selections get a route-derived box.
Sync shape coordinates over WebSockets	DONE	frontend/src/canvas/shapeDoc.js	Every shape is a Y.Map so per-field edits merge (recolour + move same shape both survive).
Undo / Redo (Y.UndoManager)	DONE	frontend/src/components/Canvas.jsx	Create, delete, move, resize, recolour, text edits, connector re-wiring and bends. Ctrl+Z / Ctrl+Y.
Connector Arrow tool	BONUS	frontend/src/canvas/connectors.js frontend/src/canvas/ConnectorNode.jsx	A full connector SYSTEM: smart shape attachment, 3 routing modes, bend points, 6 head styles, 12 presets, live re-routing. See Milestone 3.
Throttle cursor emits (~30-60/sec)	OPTIONAL	—	Still open. Cursor moves still emit on every mousemove — confirmed unchanged in Canvas.jsx. Shape drags ARE throttled at 45ms; the same pattern still applies cleanly to cursors when you scale to more users.

Task	Status	File / Evidence	What is left / Notes
Zoom / pan viewport	DONE	frontend/src/components/Canvas.jsx	Wheel zooms to the pointer (0.2x-4x), space+drag pans, footer - /%/+ controls. The camera is LOCAL per user; document coordinates stay world coordinates.

Verdict: COMPLETE, and well past the brief. The whiteboard has 16 shapes plus a full connector system, selection/resize/rotate/move/delete, styling and text-formatting panels, clipboard, locking, layer ordering, zoom/pan, PNG export, remote selection awareness, and Undo/Redo. Field-level CRDT merge is proven by test-shapes.mjs; the connector layer by test-connectors.mjs (28 checks).

Mid-Project Review — The Audit

This is not coding work. It is proof. Axler0 asks you to demonstrate, not to assert.

Task	Status	File / Evidence	What is left / Notes
SYNC AUDIT - draw in Tab A, must appear instantly in Tab B	VERIFY	frontend/src/components/Canvas.jsx	Mechanism proven headlessly for shapes (11/11) and connectors (28/28). Still RECORD the two-tab demo - now with connectors re-routing live, code running through a remote provider, chat messages arriving, and a second user getting REJECTED at the door.
Prove concurrent edits MERGE (no lost keystrokes / no clobber)	DONE	frontend/test-shapes.mjs backend/test-workspace.mjs	test-shapes.mjs proves two users editing the SAME shape (one recolours, one moves) both survive. 11 assertions pass.
ISOLATION AUDIT - room alpha must not leak into room beta	DONE	backend/test-workspace.mjs	Structurally guaranteed: workspace is read from the token. 15/15 pass. The execute endpoint and the new /api/auth endpoints both sit behind their own auth walls (workspace token vs. account token) — tested.
AWARENESS CHECK - named cursors + selections render on the CANVAS	DONE	frontend/src/components/Canvas.jsx	Cursors AND coloured remote selection boxes with names.
AWARENESS CHECK - remote selections render inside the EDITOR	VERIFY	frontend/src/components/Editor.jsx	y-monaco via MonacoBinding (preserved through the IDE rewrite and the AI-assistant integration). Confirm you SEE the coloured selection in the code pane.

Task	Status	File / Evidence	What is left / Notes
LATE JOINER - a third window gets the existing canvas	DONE	backend/services/socketService.js frontend/test-shapes.mjs	Server sends full Y state on join; test-shapes.mjs confirms a fresh doc receives the whole canvas. The shared run console and chat history persist the same way.

The 60-second isolation test — run this today

1. Window 1 → room "alpha". Draw a big scribble.
2. Window 2 → room "beta". It must be COMPLETELY BLANK.
3. Draw in beta. Window 1 must not change by a single pixel.
4. Window 3 → room "alpha". It must immediately show window 1's scribble.

Step 2 or 3 leaking = broken room logic. Step 4 working = late-joiner sync, which is a bonus you can claim.

Verdict: AUDITED HEADLESSLY (278 checks across eight standing suites — see the Session Replay section — plus an 8-check stress suite), NOT YET AUDITED VISUALLY. The two-tab demo now has its best story yet - connectors gliding, code running on a remote sandbox, chat messages landing live, and a second user getting REJECTED at the door. Record it.

Week 3 — Persistence Layer & Code Editor Sync

Goal: Yjs state is periodically written to MongoDB as binary so sessions survive restarts; Monaco is bound directly to a Yjs shared string.

Task	Status	File / Evidence	What is left / Notes
Connect the backend to MongoDB	DONE	backend/config/db.js	Degrades to memory-only if MONGO_URI is blank. Atlas needs only a .env change - no code edits. The same flag now also switches the account store (userStore.js) to Mongo.
Periodically save the BINARY Yjs state to Mongo	DONE	backend/services/socketService.js backend/models/DocState.js	Debounce 2s snapshot. Buffer, not JSON. The snapshot now also carries the IDE's shared language, run history, and chat message log, since they all live in the same doc.
Sessions survive a server restart (recovery)	DONE	backend/services/socketService.js	getRoom() rehydrates via Y.applyUpdate on first join.
Verify the snapshot in MongoDB Compass	VERIFY	backend/models/DocState.js	BLOCKED: MongoDB 8.x does not run on Windows 10. Use Atlas, then screenshot the binary state field for your report.
users collection (Blueprint 8.1)	DONE	backend/models/Workspace.js backend/models/User.js	DESIGN CHOICE, revisited: workspace MEMBERS remain embedded in the workspace document (no cross-workspace identity needed)

Task	Status	File / Evidence	What is left / Notes
			there). A SEPARATE, optional User collection was added for the account layer — see Milestone 5 — and the two are linked only by a shared <code>userId</code> , never merged. Be ready to defend both halves of this choice.
rooms collection with owner + allowedUsers (Blueprint 8.2)	DONE	backend/models/Workspace.js backend/services/workspaceStore.js	Workspace = room. <code>adminId</code> is the owner; <code>members[]</code> is allowedUsers. Repository runs on Mongo OR in-memory behind one API.
updatelogs collection (Blueprint 8.4)	DONE	backend/models/UpdateLog.js backend/services/updateLogService.js	{ <code>roomId</code> , <code>seq</code> , <code>payload</code> : Buffer, <code>userId</code> , <code>username</code> , <code>timestamp</code> }. Same dual-mode contract as <code>workspaceStore</code> . <code>seq</code> is allocated ATOMICALLY (seed once behind a shared promise, then a synchronous read-and-increment). Now also COALESCES same-user bursts within 400ms — see the Session Replay section for the full anti-flood fix.
Integrate Monaco Editor into React	DONE	frontend/src/components/Editor.jsx frontend/src/monaco-setup.js	Single-instance fix via <code>loader.config({ monaco })</code> + Vite workers. Preserved through the IDE rewrite and the AI-toolbar integration.
Bind the Monaco text model to a Yjs shared string	DONE	frontend/src/components/Editor.jsx	<code>MonacoBinding(ytext, model, editors, awareness)</code> . Still the one shared code buffer the whole IDE — and now the AI assistant — builds on.
Language switcher / multiple files	DONE	frontend/src/components/Editor.jsx backend/services/execution/languages.js	Five languages (JavaScript, Python 3, Java, C, C++). The choice lives in <code>Y.Map('editorMeta')</code> , so the dropdown AND Monaco's syntax mode switch for EVERYONE together. The dropdown is populated from the server's actual provider-backed catalog.

Verdict: DONE IN FULL. The `rooms/users` schemas landed as `models/Workspace.js` and `models/User.js`, binary snapshots and the `MonacoBinding` both work, the editor supports five languages, and `updatelogs` ships behind the same dual-mode repository pattern as everything else, now with burst-coalescing. NOTE: local MongoDB is impossible on your Windows 10 machine; Atlas is a one-line `.env` change, and `replay/accounts` both work identically in memory mode.

Week 4 — Access Control & Replay

Goal: JWT auth, users can only join rooms they were invited to, and a replay slider that scrubs backward through session history.

Task	Status	File / Evidence	What is left / Notes
Password hashing (bcrypt)	DONE	backend/services/workspaceService.js backend/models/Workspace.js	Workspace secret code hashed with bcrypt (cost 10). The hash never leaves the server - publicView() strips it. Account passwords (Milestone 5) use the identical bcrypt(10) pattern in authService.js.
Register / login endpoints	DONE	backend/routes/workspaceRoutes.js backend/controllers/workspaceController.js	Implemented as POST /workspaces (create) and POST /workspaces/:id/join. Workspace-scoped auth, not global accounts — a SEPARATE, optional pair of endpoints (POST /api/auth/signup, /login) now exists one layer up for the account system; see Milestone 5.
Issue + verify JWT	DONE	backend/utils/token.js	THREE token kinds now: an ACCESS token (workspace member), a LOBBY ticket (waiting), and a USER token (account identity, kind: 'user'). Each kind is checked explicitly wherever it is required, so a token minted for one purpose can never be replayed as another.
AUTHENTICATE THE SOCKET HANDSHAKE	DONE	backend/services/socketService.js	io.use() verifies the token before ANY event handler is registered. A socket with no token is refused outright. Tested.
Workspace creation endpoint + secret code	DONE	backend/controllers/workspaceController.js	Creator becomes sole admin. Unique WS-XXXXXX ID generated with an unambiguous alphabet. Creating while signed in now also links the workspace to your account (optionalUser middleware) without changing anonymous creation at all.
ENFORCE: user can only join a workspace they are allowed into	DONE	backend/services/socketService.js backend/middleware/authMiddleware.js	THE HEADLINE WEEK 4 DELIVERABLE. Membership is re-checked on EVERY socket connect, so a removed user is locked out instantly. The same middleware family now

Task	Status	File / Evidence	What is left / Notes
			guards code execution AND distinguishes workspace-membership auth (requireMember) from account auth (requireUser).
Access policy: PERMISSION BASED (admin approves each joiner)	DONE	backend/services/workspaceService.js frontend/src/pages/WaitingRoom.jsx	Correct password is NOT enough. Joiner is parked in a lobby until the admin decides.
Access policy: JOIN USING PASSWORD (immediate entry)	DONE	backend/services/workspaceService.js	Correct code, straight in. No waiting room.
Admin can SWITCH policy at runtime	DONE	frontend/src/components/AdminPanel.jsx backend/services/workspaceService.js	Existing collaborators stay connected. Only future joiners follow the new rule; anyone already waiting stays waiting.
WAITING ROOM with live auto-update (no refresh)	DONE	frontend/src/pages/WaitingRoom.jsx	A lobby socket has NO sync handlers registered at all - so a waiting user cannot read the doc, see cursors, or enumerate peers. Tested.
Admin Join Requests panel (username, time, approve/reject)	DONE	frontend/src/components/AdminPanel.jsx	Approval PUSHES an access token down the waiting socket. No polling.
Remove a user (kick)	DONE	backend/services/workspaceService.js frontend/src/components/AdminPanel.jsx	Their sockets are force-disconnected and their token dies on the next connect.
Privilege escalation blocked	DONE	backend/services/socketService.js backend/test-workspace.mjs	Admin authority is read from the SIGNED token, never from the event payload. A member emitting admin:approve gets an error. Tested.
Input validation, sanitisation, rate limiting	DONE	backend/utils/validate.js	10 join attempts per IP per workspace per minute. "No such workspace" and "wrong code" return an IDENTICAL message, so the endpoint is not an ID oracle. The execute endpoint and the new /api/auth/login endpoint each add their own per-scope rate limit.
Login / Create / Join pages	DONE	frontend/src/pages/Landing.jsx frontend/src/pages/CreateWorkspace.jsx frontend/src/pages/JoinWorkspace.jsx	Landing offers Create or Join as before, plus an optional Sign in / Sign up path into the account

Task	Status	File / Evidence	What is left / Notes
	DONE		system (Milestone 5) that never gates the anonymous flow.
Store token, protect routes, attach token to socket		frontend/src/utils/session.js frontend/src/utils/socket.js	sessionStorage, so two tabs can be two different users - which is how you demo it. The same workspace session token authorises Run and Chat; a separate account token (useAuth.js) governs the Dashboard.
List MY workspaces / invite by name	DONE	backend/models/User.js backend/services/authService.js backend/services/userStore.js backend/routes/authRoutes.js frontend/src/hooks/useAuth.js frontend/src/pages/Auth.jsx frontend/src/pages/Dashboard.jsx	SHIPPED — reversing the earlier call. An optional User model gives an account a stable identity across workspaces via a denormalised workspaces[] index (role + workspaceId), refreshed via addMembership() on create/join. GET /api/auth/me returns the enriched list; Dashboard.jsx renders it and re-enters a workspace with one click, re-checking membership server-side rather than trusting the client. Anonymous create/join is completely unaffected — this sits strictly alongside it.
Log every update to updatelogs (for replay)	DONE	backend/services/socketService.js	Appended AFTER the relay, on purpose: collaborators never wait on a database write. appendUpdate() now also coalesces same-user bursts inside a 400ms window before writing — see Session Replay.
Socket events get-replay-logs / replay-logs	DONE	backend/services/socketService.js frontend/src/hooks/useCollaboration.js	Streamed as replay-logs-begin / -chunk / -end (+ error with specific codes), each chunk packing payloads into ONE buffer (max 400 entries / 512 KB), which is why the naive one-packet reply used to die past ~10 updates. Registered inside the member branch only, so a lobby socket structurally cannot read history.
REPLAY: scrub backward through session history	DONE	frontend/src/components/ReplaySlider.jsx frontend/src/canvas/replay.js	A second, read-only Y.Doc is rebuilt from the log prefix, drawn by the SAME

Task	Status	File / Evidence	What is left / Notes
			renderer as the live board, and replays the whiteboard AND the Monaco editor together. Play/pause, stepping, 0.5-4x speeds, fit/zoom, keyboard control, forward-only cache with bounded checkpoints.

Verdict: ACCESS CONTROL IS DONE, and the identity model is now two clean layers instead of one. Workspace auth (JWT, handshake authentication, passwords, admin approval, waiting room, runtime policy switching, kick, rate limiting) is unchanged and still airtight; an OPTIONAL account layer sits above it for people who want a dashboard, and neither layer can be used to bypass the other. Replay ships with a materially safer history budget. Week 4 — and the entire Axlero week plan — remains COMPLETE.

Milestone 3 — Connector System & Executable IDE (17 July)

Two deliverables beyond the Axlero week plan: a draw.io-grade connector/arrow system on the whiteboard, and a multi-language executable IDE in the code pane. Both were fully collaborative over the Yjs + Socket.io pipeline at the time — zero changes to the sync protocol. The execution half was later re-platformed onto remote providers; see Milestone 5.

Task	Status	File / Evidence	What is left / Notes
Connector data model - one record type, endpoints reference shapes	BONUS	frontend/src/canvas/connectors.js frontend/src/canvas/shapeDoc.js	THE core idea: attached endpoints store a shapeld and their position is DERIVED at render time. Move a shape and every attached connector re-routes on the next frame - locally, remotely, with ZERO extra network traffic.
Smart edge attachment + snapping	DONE	frontend/src/canvas/connectors.js	N/E/S/W anchor dots on every shape (rotation-aware) plus "auto" attachment to the shape's REAL outline. 14px anchor snap, 8px body pad, green ring + anchor preview while wiring.
Three routing modes	DONE	frontend/src/canvas/ConnectorNode.jsx	Straight / elbow (axis-aligned zig-zag) / curved (Catmull-Rom spline). Convert between them any time; Straighten clears bends, Reverse flips direction.
Arrowheads, styles & 12 presets	DONE	frontend/src/canvas/shapes.jsx frontend/src/components/Toolbar.jsx	Six head styles on either end; dashed, dotted, thick, thin, double presets; C and A toolbar tools.
Bend-point editing	DONE	frontend/src/components/Canvas.jsx	Round endpoint handles, square waypoint handles, midpoint dots that birth a bend, double-click to insert one.

Task	Status	File / Evidence	What is left / Notes
Live drag streaming	BONUS	frontend/src/canvas/shapeDoc.js	updateShapeLive(): 45ms-throttled commits under an undo-invisible "live" origin. A whole drag still undoes as ONE step.
Object-system verbs for ALL shapes	BONUS	frontend/src/components/Canvas.jsx frontend/src/components/PropertyPanel.jsx	Copy / paste / duplicate (Ctrl+C/V/D) with connector re-wiring, Ctrl+A, lock, bring forward / send backward, PNG export.
Sandboxed execution service	DONE	backend/services/execution/runner.js backend/services/execution/providers/local.js	The ORIGINAL local runner: temp dir -> write source -> [compile] -> run (stdin piped) -> cleanup, with a hard SIGKILL wall timeout, output caps, POSIX ulimits. Still shipped as the 'local' provider adapter — an unsandboxed-toolchain fallback rather than the default path, per Milestone 5.
Language registry: 5 languages	DONE	backend/services/execution/languages.js	JavaScript (Node), Python 3, Java, C, C++. Each entry now also carries the identifiers Judge0/Piston/paiza expect, resolved at runtime — see EXECUTION.md.
Execute API behind real auth	DONE	backend/routes/executeRoutes.js backend/server.js	POST /api/workspaces/:id/execute (+ GET .../languages), guarded by the SAME requireMember as every other workspace route. Per-user rate limit; JSON body limit raised for whole source files.
IDE frontend	DONE	frontend/src/components/Editor.jsx	Language dropdown, Run with busy state (Ctrl+Enter), collapsible stdin panel, resizable console with per-run status/duration/exit code, Monaco find/replace/go-to-line/folding. Now shares the pane with the AI Toolbar — see Milestone 5.
Shared console + shared language	BONUS	frontend/src/components/Editor.jsx	Every run result is appended to Y.Array('runHistory') (capped at 20), so ALL collaborators see the same console, live, surviving reloads via the same Mongo snapshot. Language choice is shared via Y.Map('editorMeta').

Task	Status	File / Evidence	What is left / Notes
Tests: connector geometry, CRDT & execution	DONE	frontend/test-connectors.mjs backend/test-execute.mjs	connectors: 28 checks bundling the real shipped modules via esbuild; execution: originally 76 checks against real processes, now 100 checks run against a mock provider API that speaks Judge0/Piston/paiza's real wire formats — see Milestone 5.

Verdict: BOTH PARTS SHIPPED, tested end-to-end at the time. The execution suite caught two genuine sandbox bugs before any user could (dash rejecting ulimit -u; ulimit -v OOM-killing V8/JVM at startup) — the argument for testing sandboxes with real processes, not mocks. That local sandbox is preserved as a fallback provider; the default execution path has since moved off it entirely (Milestone 5). MILESTONE3.md in the repo remains the viva-ready writeup of the data model, geometry and original sandbox design.

Milestone 4 — Pen / Brush Studio, True Eraser & Robust Undo (22 July)

The fourth build turns the placeholder Pen into a full drawing tool and adds the two things a collaborative whiteboard is judged on - a real eraser and dependable Undo. All of it is additive on the existing Yjs + Socket.io pipeline with zero changes to the sync protocol: a stroke is still one ordinary record in the same shapes array, so a brush, a partial erase and an undo all ride the paths that already worked.

Task	Status	File / Evidence	What is left / Notes
Brush customization panel	BONUS	frontend/src/components/BrushPanel.jsx	7 brush styles, a colour palette + picker, recent colours, thickness 1-60, opacity, smoothing and pressure toggles, live preview. Settings persist to localStorage.
Seven brush styles	DONE	frontend/src/canvas/brushes.js frontend/src/canvas/ShapeNode.jsx	Pen, Pencil, Marker, Highlighter, Calligraphy, Dashed, Dotted — one path record rendered through a single brush-aware switch.
Smooth stroke rendering	DONE	frontend/src/canvas/brushes.js	Chaikin corner-cutting over an RDP-simplified path. RDP also shrinks the stored point list, so sync and redraw stay cheap.
True partial ERASER	BONUS	frontend/src/canvas/brushes.js frontend/src/components/Canvas.jsx	Rubs out only the part of a stroke you drag across; the rest survives as independent strokes. Adjustable size, hover cursor, sampled drag, bounding-box broad-phase.
Erase = one atomic transaction	DONE	frontend/src/canvas/shapeDoc.js	applyErase() deletes the original stroke and pushes one path per surviving run in a SINGLE transaction: one

Task	Status	File / Evidence	What is left / Notes
			undo step, one atomic sync message.
Undo / Redo across every action	DONE	frontend/src/components/Canvas.jsx frontend/src/components/Toolbar.jsx	Ctrl+Z, Ctrl+Shift+Z and Ctrl+Y plus visible toolbar buttons. Only local (origin-null) writes are tracked, so a collaborator's edits are never reverted by your undo.
Live-drawing performance	BONUS	frontend/src/components/Canvas.jsx frontend/src/canvas/ShapeNode.jsx	The in-progress stroke lives in local state + awareness and is written to the ydoc ONCE on release. ShapeNode is React.memo-wrapped.
Real-time sync - protocol unchanged	DONE	frontend/src/hooks/useCollaboration.js	New strokes, brush settings, partial-erase results and undo/redo all travel over the EXISTING relay as ordinary shape records or awareness fields.
Tests: brush / eraser / undo suite	DONE	frontend/test-brushes.mjs	Originally 29 checks; now 43 (see the Session Replay section for the current cross-suite total). Existing suites untouched and still green; Vite build clean.

Verdict: ALL FOUR ASKS SHIPPED. A clean, modern Pen/Brush panel, a true partial Eraser that keeps the rest of the stroke, and Undo/Redo covering every operation. The eraser's atomic delete-and-re-split is the load-bearing idea. DRAWING_TOOLS.md in the repo is the viva-ready writeup.

Milestone 5 — Accounts, Remote Execution, AI Assistant & Team Chat (13 August)

Four deliverables beyond the Axlero week plan, none of which touch the sync protocol or the workspace security model: an optional account/dashboard layer, code execution re-platformed onto remote sandboxes, an AI coding assistant in the editor pane, and a real-time team chat panel. A structural React error boundary also landed underneath all of it.

Task	Status	File / Evidence	What is left / Notes
Account model, separate from workspace membership	DONE	backend/models/User.js backend/services/userStore.js	One optional document per signed-up person: userId, email, username, passwordHash, and a denormalised workspaces[] index (workspaceId + role) so the dashboard never has to scan every workspace document. Workspace.members remains the sole source of truth for authorisation — this collection is just an index on top of it, linked by one shared userId.

Task	Status	File / Evidence	What is left / Notes
Signup / login / me endpoints	DONE	backend/routes/authRoutes.js backend/services/authService.js	POST /api/auth/signup, POST /api/auth/login, GET /api/auth/me. bcrypt(10) hashing, a rate-limited login (12/min per email), and a dedicated USER-kind JWT that a workspace route will never accept.
requireUser / optionalUser middleware	DONE	backend/middleware/authMiddleware.js	requireUser guards account-only routes (the dashboard); optionalUser lets workspace create/join attach an account without ever rejecting an anonymous request. The two identity systems are structurally kept from colliding — a workspace access token and a user token are different JWT kinds and neither middleware accepts the other.
Sign in / Sign up pages + auth hook	DONE	frontend/src/pages/Auth.jsx frontend/src/hooks/useAuth.js	useAuth.js owns the account token in isolation from the per-workspace session (utils/session.js), so being logged into an account and being a member of a workspace remain two independent facts about the same browser tab.
Dashboard: list + one-click re-open of every workspace	DONE	frontend/src/pages/Dashboard.jsx	Reverses the earlier "optional, would need a global identity" call from Week 4. Calls api.enterWorkspace(), which re-checks membership server-side — the dashboard shortcuts typing a room ID and a password, never the authorisation check itself.
Execution re-platformed onto remote providers	DONE	backend/services/execution/providers/judge0.js backend/services/execution/providers/piston.js backend/services/execution/providers/paiza.js backend/services/execution/providers/index.js	Judge0 CE is now the default primary provider, with Piston and paiza.io as fallbacks in a configurable chain (EXEC_PROVIDERS). The original local sandbox (Milestone 3) remains as the 'local' adapter — present but excluded from the default chain — so the project no longer assumes a compiler, JDK or Docker install exists on the host at all. See EXECUTION.md for the full provider comparison and rationale.
Provider-agnostic	DONE	backend/services/execution/index.js	Unchanged responsibility, new shape: queueing, a

Task	Status	File / Evidence	What is left / Notes
execution orchestrator	DONE		concurrency slot pool, provider probing/caching, and fallback across the chain, all speaking the ONE canonical result shape from result.js so the UI never sees a Judge0 status.id or a Piston signal directly.
OpenAI-backed AI coding assistant		backend/services/ai/aiService.js backend/services/ai/prompts.js backend/routes/aiRoutes.js	POST /api/ai handles nine actions — chat, explain, generate, error, debug, tests, optimize, convert, document — each with its own system prompt. With no OPENAI_API_KEY configured it returns a clearly-labelled mock response instead of failing, so the rest of the app is demoable without a key.
AI request validation		backend/services/ai/validators.js	Action allow-list, and length caps on message/code/error (8000 / 30000 / 8000 chars) enforced before anything reaches the model.
AI assistant frontend		frontend/src/ai/AIPage.jsx frontend/src/ai/AIChat.jsx frontend/src/ai/AIToolbar.jsx frontend/src/ai/aiApi.js	A dedicated /workspace/:id/ai route with a nine-action toolbar and a chat-style transcript (AIMessage.jsx) that can read the current code/language/error context from the shared editor state.
Workspace team chat panel		frontend/src/components/ChatPanel.jsx	Messages live in ydoc.getArray('chatMessages') (capped at 100) and typing state in awareness, so chat rides the EXISTING Yjs + Socket.io relay with zero new backend code and persists in the same Mongo snapshot as everything else.
App-level error boundary		frontend/src/components/ErrorBoundary.jsx	React 18's createRoot unmounts the ENTIRE tree on an uncaught render error — which is exactly how one undeclared identifier in PropertyPanel used to take the whole whiteboard down. That bug is fixed, but the class of failure is now structurally contained: a bad record degrades to "that one object doesn't draw" instead of a blank page.
Toast notifications		frontend/src/components/Toast.jsx frontend/src/hooks/useToasts.js	A small shared toast queue used across the workspace

Task	Status	File / Evidence	What is left / Notes
			shell for non-blocking status messages (join/kick/policy changes, AI/exec errors) instead of alert().
New execution test suite (provider adapters)	DONE	backend/test-execute.mjs backend/test-support/mock-execution-api.mjs	Rewritten around a mock server that speaks Judge0/Piston/paiza's real wire formats and genuinely compiles and runs the submitted program underneath, so the adapters are tested against real gcc diagnostics and real Unicode without needing network access or an API key in CI. Grew from 76 to 100 checks; verified passing for JavaScript, Python, C and C++ in this environment (Java requires a JDK on the host running the tests).

Verdict: ALL FOUR DELIVERABLES SHIP, and — this is the part worth saying explicitly in a review — none of them required touching `socketService.js`'s `sync` path, the workspace auth model, or the replay pipeline. Accounts are an index on top of workspaces; execution swapped its backend without changing its API shape; chat and the AI assistant both reuse the existing Yjs relay and REST/auth patterns respectively. `EXECUTION.md` documents the provider comparison and architecture end-to-end.

Final Review — Deployment, Testing, Polish

Task	Status	File / Evidence	What is left / Notes
MongoDB Atlas cluster (cloud DB)	VERIFY	backend/.env	STILL THE ONLY VIABLE PATH: MongoDB 8.x will not run on Windows 10. Migration is one .env line — the store layer for workspaces, replay logs AND accounts all speak both persistence modes behind the same flag.
Deploy backend (Render)	OPTIONAL	—	Set <code>CLIENT_ORIGIN</code> to the Vercel URL, <code>JWT_SECRET</code> to a long random string, and (optionally) <code>OPENAI_API_KEY</code> . NOTE: remote execution no longer requires the Render box to have any compiler toolchain installed — Judge0/Piston/paiza run the code, not the host — which removes what used to be the trickiest deployment requirement.
Deploy frontend (Vercel)	DONE	frontend/src/utils/socket.js	The hard-coded <code>localhost:5000</code> is gone. <code>SERVER_URL</code> reads <code>VITE_SERVER_URL</code> .
Unit / integration tests	BONUS	frontend/test-connectors.mjs backend/test-execute.mjs	GREW again: 278 checks across eight standing suites (workspace

Task	Status	File / Evidence	What is left / Notes
		frontend/test-shapes.mjs backend/test-workspace.mjs frontend/test-brushes.mjs backend/test-replay.mjs frontend/test-replay.mjs backend/test-updatelog.mjs backend/stress-replay.mjs	15, execute 100 — up from 76, backend replay 27, the NEW updatelog suite 16, shapes 11, connectors 28, brushes 43 — up from 29, frontend replay 38), plus the 8-check replay stress suite. Verified by running them: workspace, backend replay, frontend replay, shapes, connectors, brushes and updatelog all pass in full (15/15, 27/27, 38/38, 11/11, 28/28, 43/43, 16/16, 8/8); execute passes for every language this sandbox has a toolchain for (Java needs a JDK on the test host).
Resilience: drop the network, keep drawing, reconnect	OPTIONAL	—	Great demo moment - Yjs merges the offline edits on reconnect. Still untested.
README / project report	DONE	MILESTONE3.md DRAWING_TOOLS.md QA_PASS.md REPLAY.md EXECUTION.md SyncSpace_Report.pdf	The standalone README.md was folded into the growing SyncSpace_Report.pdf. EXECUTION.md is new: it documents the provider comparison (Judge0 vs. Piston vs. paiza vs. commercial options), the orchestrator architecture, and the canonical result shape end-to-end. MILESTONE3.md, DRAWING_TOOLS.md, QA_PASS.md and REPLAY.md are all still present and unchanged.
dev.bat one-click launcher	BONUS	dev.bat	Unchanged: validates Node/npm + folder structure, auto-installs missing node_modules, creates .env from .env.example when needed, waits for the backend before starting the frontend, then opens the browser.

Known weaknesses worth fixing before any review

Unthrottled cursor emits. Still true — confirmed against the current Canvas.jsx. Canvas.jsx writes cursor positions to awareness on every single mousemove. With two users this is invisible; with six it will stutter. Shape DRAGS are throttled at 45ms, and the update-log coalescing added in this milestone uses the identical windowed-merge idea — applying the same two-line throttle pattern to cursors is a strong thing to be able to explain, and is now backed by a second working example of the pattern elsewhere in the codebase.

Process-level sandbox — now the fallback path, not the default. The original weakness (temp dirs, timeouts and ulimits instead of container isolation) still describes the `local` provider exactly, but that provider is no longer in the default chain: an ordinary Run now leaves this server's process space entirely and executes on Judge0's cgroups/namespaces sandbox (or Piston/paiza). The residual risk is narrower — it only applies if EXEC_PROVIDERS is explicitly configured to include `local` — and is worth stating precisely that way rather than as an open weakness of the shipped default.

AI assistant has no per-user rate limit of its own yet. The execute endpoint and the login endpoint are both rate-limited; POST /api/ai currently is not, beyond the input-length caps in validators.js. Worth adding the same per-user token-bucket pattern before a public deployment, since OpenAI usage is metered.

The RED rows are gone, and the new features didn't reopen any. updatelogs -> replay events -> ReplaySlider all shipped, tested and documented at Week 4; accounts, remote execution, the AI assistant and chat all shipped this milestone without touching the sync protocol or the security model. Everything on the board is additive work on a foundation that runs, replays, and now also authenticates, executes, and assists — all as options on top of the same core.

Session Replay — update: coalescing & a raised ceiling (13 August)

The 27 July build already had a correct streaming transport and a safe 5,000-update cap. The open risk flagged at the time was that a single continuous drag emits roughly 60 updates a second, so a few minutes of ordinary dragging could burn through a meaningful share of a room's whole replay budget. That risk is now closed at the source, not just pushed further out with a bigger number.

Burst coalescing. `appendUpdate()` now checks, before writing a new log entry, whether the previous entry came from the SAME user within a 400ms window (`COALESCE_WINDOW_MS`) and is small enough to still merge (under 16KB, `MAX_COALESCED_BYTES`). If so, `Y.mergeUpdates()` folds the new update into the existing entry instead of appending a new one. `Y.mergeUpdates(a, b)` is defined to produce an update equivalent to applying a then b, so a PREFIX of the coalesced log is still exactly the document as it stood at that point — the one property the whole replay design depends on, and it survives merging by construction, not by convention.

What this buys in practice. A drag that used to cost ~60 log entries per second of dragging now costs a small, roughly constant number of entries regardless of how long the drag lasts, because consecutive samples from the same gesture keep folding into the same entry. Separate deliberate actions still stay separate: the burst window resets on every merge attempt rather than refreshing, so a continuous drag cannot coalesce forever into one giant entry, and two updates from different users are never merged together even if they land in the same instant.

The ceiling moved too, because it can afford to. `MAX_LOGS_PER_ROOM` went from 5,000 to 50,000 entries, and a second, independent `MAX_LOG_BYTES_PER_ROOM` ceiling (32MB of stored payload) was added alongside it — whichever limit is hit first wins, so neither a flood of tiny updates nor a handful of enormous ones can run away unchecked. Because the streamed transport already didn't care about total session length (chunks stay bounded at 400 entries / 512KB regardless), raising the entry ceiling costs only per-room memory/storage, not a slower or riskier replay.

New test suite. `backend/test-updatelog.mjs` (16 checks, all passing) exercises this directly and in isolation: a 300-update simulated drag stores far fewer log entries than updates while still reconstructing the exact final state and every valid intermediate prefix; actions separated by a pause, or from different users, are proven to stay separate steps; the entry and byte ceilings are each proven to stop recording independently and report 'capped'; and a malformed or empty update is proven to degrade gracefully rather than throw, on the same hot path every collaborator's edits pass through.

Suite totals, current

278 checks across eight standing suites — backend: workspace 15, execute 100 (was 76), replay 27, updatelog 16 (new); frontend: shapes 11, connectors 28, brushes 43 (was 29), replay 38 — plus the 8-check replay stress suite (`backend/stress-replay.mjs`). All green with MongoDB in memory-mode and a full local toolchain; verified directly for every suite except `execute`'s Java path, which needs a JDK on whichever machine runs the tests.

The history cap — `MAX_LOGS_PER_ROOM = 50,000`

What it is now. Two constants in `backend/services/updateLogService.js`, checked in this order: the entry count (`MAX_LOGS_PER_ROOM`, 50,000 — up from 5,000) and the stored byte total (`MAX_LOG_BYTES_PER_ROOM`, 32MB). When a room's history reaches either ceiling, recording STOPS, the capped flag is set, and the replay dialog shows an honest "history full" chip. The head is kept and the tail dropped — never the reverse — because Yjs updates form a causal chain: an update that edits a shape references the update that created it, so a PREFIX of the log is always a complete, consistent document while a suffix is not.

What 50,000 means in practice, now that bursts coalesce. Time is still not the unit that matters — update COUNT and stored bytes are. But the count itself now grows much more slowly during the expensive case: a continuous live drag, which used to

log at ~60 entries/second, now folds into a handful of coalesced entries per burst instead. A whole freehand stroke, a shape, a connector or a text block are still one update apiece; editor typing still batches keystrokes into small updates. The practical effect is that the old worked example — "one change every 2 seconds for 5 hours hits the cap at 2h 47m" — no longer represents the worst case at all: continuous dragging, the scenario that used to burn the budget fastest, is now the cheapest kind of activity to log.

Raising it further is still one line, and the transport still doesn't care. Chunks stay bounded at 400 entries / 512KB whatever the session length, the server yields between chunks, and the client shows progress for both the download and the prepare phase. The costs of a larger cap remain per-room memory/storage and a slightly longer, non-blocking "Preparing replay..." phase — smaller costs than before, since coalescing means a given cap now covers proportionally more real collaboration time.